For equality, Ruby has a couple of different versions, though the one you want is almost always ==. It's the version that compares on the value of the object, not the identity of the object.

Finally, avoid using assignment in the conditional, `if x = gets` (even though there are some examples in this book that do this for compactness or clarity). If you do, it's probably a good idea to put parentheses around the conditional, like this: `if (x = gets)`, to draw attention to the unusual code.

Data

When creating an array or a hash, it is better to use the literal syntax [] or {} over `Array.new` or `Hash.new`. If applicable, use `%w` for an array of short strings or `%i` for an array of symbols.

In general, prefer symbols as Hash keys, or at the very least, some other immutable object. When using a symbol as a key, you should use the newer `{foo: bar}` syntax rather than `{:foo => :bar}`. (This is a declaration against interests; we personally love the way the Hashrocket looks, but objectively, more people find the shorter syntax easier.)

Ruby has a handful of pre-existing methods that combine iteration functions, like `flat_map` and `reverse_each`. These tend to be faster than chaining the two methods, so you should use these when appropriate.

For Strings, use interpolation syntax rather than concatenating strings with bits of stuff, so `"#{last_name}, #{first_name}"` not `last_name + ", " + first_name`. We prefer using double-quoted strings in all cases on the grounds that it's easier to modify if you later add interpolation to the string—double-quoted strings are not appreciably slower than single-quoted strings. If the string itself includes double quotes, use the `%Q` syntax. For long strings use heredocs, especially the `<<~SQUIGGLE` syntax that manages indentation.

Similarly, use `%r` for regular expressions that contain / characters. Consider using `x` as a modifier for complex regular expressions to allow for more expressive spacing. Use a comment on regular expression literals to point to Rubular or something similar where a reader can try the regular expression.

The Ruby style guides offer a lot of other suggestions; this is only some of what a full style guide might include.

Using RuboCop

When you write Ruby code, you often don't want to have to think about style. What you want to do is write your code and solve your problem. However, what usually happens is that once you have your team review your code, people on your team start to nit-pick your minor style choices, sometimes contradicting each other. This is, to say the least, not compatible with great team morale.

Enter Rubocop.[1] RuboCop is a *linter*, which means that it automatically checks your code for style and then either flags discrepancies or optionally auto-corrects them. (The name "linter" comes from the first such program, `lint`, written in 1978 by Stephen C. Johnson for C code and named by analogy to a lint trap as a thing that catches small issues.)

1. https://rubocop.org